MODULE 07 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

DynamoDB I – Keys, Partitions and Indexes

The one decision you cannot change later, and the two index types the exam will not stop asking about.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 7 of 34

admin@sthithapragna.com

In a relational database you model the data and query it later. In DynamoDB you model the **queries**, and the data arrangement follows.

The vocabulary

- **Table** holds **items**, and each item is a set of **attributes**. There is no fixed schema beyond the key.
- **Partition key** decides which physical partition stores the item. Also called the hash key.
- **Sort key** is optional. It orders items that share a partition key. Also called the range key.
- **Item collection** is every item sharing one partition key value.
- Keys must be scalar: **String, Number or Binary**. Never a list, map or boolean.

> ON THE EXAM
>
> Only the key attributes need to exist on every item. Everything else varies item to item, which is what makes single-table design possible.

Attribute data types

- **Scalar:** String, Number, Binary, Boolean, Null. Only the first three can be keys.
- **Document:** List and Map, nestable up to 32 levels deep.
- **Set:** String Set, Number Set, Binary Set. Unordered, no duplicates, cannot be empty.
- Numbers hold up to 38 digits of precision, so pass them as strings when precision matters.
- Attribute names count towards the 400 KB item size, so short names save real money at scale.

> ON THE EXAM
>
> The examinable line is that keys must be String, Number or Binary. A boolean or a list can never be a partition or sort key.

Simple key or composite key?

PARTITION KEY ONLY

One item per key value

- The key must be unique per item
- GetItem by key is the access pattern
- No Query within a key
- Suits lookup tables and sessions
- Simplest possible design

PARTITION KEY PLUS SORT KEY

Many items per key value

- Uniqueness is the pair, not the key
- Query returns a range of items
- Items stored sorted by sort key
- Suits one-to-many relationships
- Enables nearly all rich patterns

THE TIEBREAKER

If a partition key value ever needs to hold more than one item, you need a sort key. Adding one later means rebuilding the table, so decide early.

How a write finds its home

01

Hash

DynamoDB hashes the partition key value.

02

Route

The hash selects one physical partition.

03

Store

The item is written, ordered by sort key within that partition.

ON THE EXAM

Everything about hot partitions, throttling and key design follows from this one mechanism. The sort key plays no part in choosing the partition.

Choosing a partition key

- **High cardinality.** Many distinct values, so data spreads across many partitions.
- **Uniform access.** Requests should be spread roughly evenly across those values.
- Good: customer ID, order ID, device ID, session ID.
- Bad: status, country, a boolean, today's date, or any fixed value.
- Cardinality alone is not enough. A million keys where one is 90% of traffic is still hot.

> ON THE EXAM
>
> Two properties, and questions test both. 'Many distinct values' is necessary. 'Accessed evenly' is what actually prevents throttling.

Hot partitions

- Each physical partition is capped at roughly **3,000 RCU** and **1,000 WCU** per second.
- That cap is **per partition**, so table-level capacity cannot rescue a single hot key.
- The symptom: **throttling while consumed capacity sits well below provisioned**.
- **Adaptive capacity** shifts throughput toward busy partitions and can split them automatically.
- Splitting takes minutes, and it cannot lift a single partition past the hard cap.

> ON THE EXAM
>
> That symptom line is the giveaway. Throttling with capacity to spare is a key design problem, never a capacity problem. Raising provisioned capacity will not help.

Write sharding

- Append a suffix to the partition key: **2026-09-04#1** through **2026-09-04#10**.
- Ten shards means ten partitions, so roughly ten times the write ceiling.
- A **random** suffix spreads writes best. A **calculated** one keeps reads targeted.
- Reads now need a query per shard, and the results merged in your code.
- Choose the shard count deliberately. Changing it later means rewriting historical keys.

> ON THE EXAM
>
> This is the standard answer for time-series data keyed by date. It trades read simplicity for write throughput, and the exam expects you to name that trade.

Four ways to model one-to-many

- **Sort key on the same partition.** One Query returns the parent and all children.
- **A nested list or map** inside the parent item. Simple, but bounded by 400 KB.
- **A GSI** where the child's foreign key becomes the index partition key.
- **Composite sort key** when the relationship is hierarchical rather than flat.
- Never a second table you join in application code. That is the pattern to avoid.

> ON THE EXAM
>
> Unbounded children means separate items. A small, bounded set that is always read with the parent can live inside the item.

Sort keys do more than sort

- Query supports **begins_with**, **between**, and the comparison operators on the sort key.
- A **composite sort key** encodes a hierarchy: **COUNTRY#STATE#CITY**.
- begins_with then queries at any level of that hierarchy with one index.
- An ISO timestamp as sort key gives you date ranges for free, since it sorts lexically.
- Sort key conditions are exact prefixes. There is no contains and no wildcard in the middle.

> ON THE EXAM
>
> Most 'I need another index' problems are really 'I need a better sort key' problems. Hierarchical sort keys serve several access patterns from one structure.

The constraints worth memorising

LIMIT	VALUE
Item size	400 KB, including attribute names
Partition key value	1 to 2,048 bytes
Sort key value	1 to 1,024 bytes
Query or Scan page	1 MB, then a LastEvaluatedKey to continue
Global secondary indexes	20 per table by default, and it can be raised
Local secondary indexes	5 per table, fixed, and only at creation
Item collection with an LSI	10 GB per partition key value

ON THE EXAM

The 400 KB limit drives the standard pattern: store large objects in S3 and keep the key in DynamoDB. The last row is a real design trap.

Secondary indexes exist for one reason: to answer a question your primary key cannot.

Global secondary indexes

- **Any attribute** can be the index partition key and sort key. Both differ from the table.
- Physically separate, with **its own capacity**, its own partitions and its own throttling.
- **Create or delete at any time**, on a live table, without downtime.
- **Eventually consistent reads only.** Strongly consistent reads are impossible on a GSI.
- Every write to the table becomes an extra write to each GSI, and you pay for it.

> ON THE EXAM
>
> GSIs are the default answer for a new access pattern. Their two real costs are eventual consistency and duplicated write capacity.

Local secondary indexes

- **Same partition key** as the table, and a different sort key. That is the whole idea.
- Stored alongside the item in the **same partition**, which is what 'local' means.
- **Supports strongly consistent reads.** The only reason to choose one.
- Shares the table's capacity rather than having its own.
- **Must be created with the table.** You cannot add or remove one afterwards.

> ON THE EXAM
>
> And the sting: any table with an LSI caps each item collection at 10 GB. Without an LSI there is no such ceiling.

GSI overloading

- Generic index keys, **GSI1PK** and **GSI1SK**, holding different meanings per item type.
- An order item might set GSI1PK to a status; a customer item might set it to an email.
- One index then serves several unrelated access patterns.
- Keeps you well inside the 20-index default quota.
- Queries must know which item type they expect, since results are mixed.

> ON THE EXAM
>
> The companion to single-table design. When you see GSI1PK in a schema, this is what is happening.

When a GSI throttles the table

- **GSI read** throttling does not affect the base table at all.
- **GSI write** throttling throttles writes to the **base table**, and to its other indexes.
- DynamoDB applies **back pressure** so the index cannot fall too far behind the table.
- So provision each GSI's write capacity at or above the table's.
- A GSI has its own partitions, so a well-spread table can still have a hot index.

> ON THE EXAM
>
> This is genuinely counterintuitive: writes to a healthy table fail because an index is short of capacity. Look for IndexWriteProvisionedThroughputExceeded.

Time-series data

- A date as partition key concentrates every write of that day on one partition.
- **Shard the key**, or use a higher-cardinality entity like device or user as the partition key.
- **A table per period** - one per month - keeps the current table small and hot.
- Old tables can then be exported and deleted cheaply, instead of deleting item by item.
- Or use **TTL** to expire old items automatically at no write cost.

> ON THE EXAM
>
> Deleting millions of expired rows with DeleteItem costs a WCU each. A table per period, or TTL, avoids that entirely.

Key design anti-patterns

ANTI-PATTERN	WHAT HAPPENS
A low-cardinality partition key	Few partitions, so throttling well below provisioned capacity
A date or timestamp as partition key	All of today's traffic hits one partition
Adding variety to the sort key to fix heat	No effect. The sort key does not choose the partition
Scanning with a filter as a query	Pays to read everything, returns a fraction
A GSI with a low-cardinality partition key	A hot index, which then throttles the base table

ON THE EXAM

Row three is the one worth remembering. Diversifying a sort key never fixes a hot partition, and it appears as a distractor.

Three ways to read, at a glance

OPERATION	WHAT IT DOES, AND WHAT IT COSTS
`GetItem`	One item by full primary key. The cheapest possible read
`Query`	Items sharing one partition key, optionally filtered by sort key. Efficient
`Scan`	Every item in the table or index. Expensive, and rarely correct

ON THE EXAM

Query needs the partition key value. If you cannot supply one, you need a different index - not a Scan.

GSI or LSI?

GLOBAL SECONDARY INDEX

Almost always this one

- Any partition key and sort key
- Added or removed at any time
- Its own capacity and throttling
- Eventually consistent only
- No size limit on collections

LOCAL SECONDARY INDEX

Only for strong consistency

- Same partition key, new sort key
- Created with the table only
- Shares the table's capacity
- Strongly consistent reads possible
- Caps item collections at 10 GB

THE TIEBREAKER

Choose an LSI only when you need strongly consistent reads on an alternate sort order, and you know it at table creation. Every other case is a GSI.

Projections

- **KEYS_ONLY** – table keys plus index keys. Smallest and cheapest.
- **INCLUDE** – keys plus a named list of attributes you choose.
- **ALL** – every attribute. Largest storage and highest write cost.
- A GSI query returning a non-projected attribute **does not fetch it**. It is simply absent.
- An LSI query can fetch non-projected attributes, at the cost of extra reads.

> ON THE EXAM
>
> The GSI behaviour catches people. An attribute missing from a query result is usually a projection problem, not a data problem.

Sparse indexes

- An item appears in an index **only if it has the index key attribute**.
- So write that attribute only on items you want indexed, and the index stays tiny.
- Classic use: an **unprocessed** flag, deleted once the item is handled.
- Querying a handful of items in a billion-item table, with no scan at all.
- Setting the attribute to null or empty does not remove it. It must be absent.

> ON THE EXAM
>
> When a scenario needs to find the small set of items in some state across a huge table, a sparse GSI is usually the intended answer.

Single-table design, briefly

- Generic key names, **PK** and **SK**, holding different meanings per item type.
- Prefixes keep types apart: **CUSTOMER#123**, **ORDER#456**, **PRODUCT#789**.
- Related items share a partition key, so one Query fetches a customer and their orders.
- That replaces the join DynamoDB does not have.
- Harder to read and to change. Not required, and not always right.

> ON THE EXAM
>
> The exam rarely asks you to design a single-table schema. It does expect you to recognise PK and SK with prefixed values and understand why they look like that.

Three of these are design questions in disguise. Ask what the **access pattern** is before you look at the options.

> An IoT platform writes sensor readings to a DynamoDB table. The partition key is the date in YYYY-MM-DD format and the sort key is a timestamp. About 40,000 readings arrive per second, spread across 50,000 devices.
>
> The table is provisioned with 60,000 WCUs but writes are being throttled heavily. CloudWatch shows consumed write capacity at roughly 5% of provisioned. Queries always ask for one device's readings over a time range.

What is the root cause, and what should the developer change?

A. Increase provisioned write capacity, since 40,000 writes per second is close to the limit

B. Change the partition key to the device ID and the sort key to the timestamp

C. Add a global secondary index with device ID as the partition key

D. Switch the table to on-demand capacity mode

◆ PAUSE AND DECIDE

What is the root cause, and what should the developer change?

A More provisioned capacity

The table already has twelve times the capacity it is consuming. Per-partition limits are the constraint, and table capacity cannot raise them.

B Device ID as partition key

Fifty thousand devices spreads writes across many partitions, and it matches the stated query pattern exactly. It fixes the cause rather than the symptom.

C GSI on device ID

Improves reads, but every write still lands on today's date partition. The write throttling is untouched, and the GSI adds write cost.

D On-demand mode

Removes capacity planning, not per-partition limits. A single hot key throttles in on-demand mode too.

THE SIGNAL

Consumed capacity at 5% of provisioned while throttling is the fingerprint of a hot partition. One date means one partition takes every write.

> An existing orders table uses customer ID as the partition key and order date as the sort key. It has been in production for two years and holds several hundred gigabytes.
>
> A new feature must list all orders with a given status, such as PENDING or SHIPPED, across all customers. Status is already stored on every order item. Eventually consistent results are acceptable, and pending orders are under 1% of the table.

How should the developer support the new access pattern?

A. Add a local secondary index with status as the sort key

B. Scan the table with a filter expression on status

C. Create a global secondary index with status as the partition key and order date as the sort key

D. Create a new table keyed by status and dual-write to both tables

◆ PAUSE AND DECIDE

How should the developer support the new access pattern?

A Add an LSI

Impossible twice over. LSIs can only be created with the table, and an LSI keeps the table's partition key, so it could never search across customers.

B Scan with a filter

A filter is applied after reading, so you pay to read hundreds of gigabytes to return under 1% of it. Slow and expensive.

C GSI on status

A different partition key on a live table is exactly what a GSI is for, eventual consistency is accepted, and the sort key gives date ordering for free.

D Second table with dual writes

Hand-building what a GSI does automatically, and introducing a consistency bug the moment one of the two writes fails.

THE SIGNAL

Two sentences settle it: the table already exists, and eventual consistency is fine. The first kills the LSI, the second removes the only reason to want one.

A document management system stores metadata in DynamoDB. Each document item holds a title, owner, tags and a full extracted text body. Text bodies range from a few kilobytes to about 12 MB.

Writes are failing intermittently for larger documents. The team needs full-text bodies retrievable by document ID, and wants to keep querying metadata efficiently.

How should the developer restructure the storage?

A Split the text body across multiple items using a chunk number in the sort key

B Compress the text body with gzip before writing it to the item

C Request an item size limit increase from AWS Support

D Store the text body in Amazon S3 and keep the object key in the DynamoDB item

◆ PAUSE AND DECIDE

How should the developer restructure the storage?

A Chunk across items

A documented pattern, but 12 MB needs about thirty items, and every read reassembles them. It also inflates the item collection considerably.

B Compress with gzip

Helps, and does not solve it. Text compressing 4:1 still leaves 3 MB, well past 400 KB. It also makes the body unreadable to queries.

C Request a limit increase

The 400 KB item size is a hard limit. No support request changes it.

D Body in S3, key in DynamoDB

The standard pattern. Metadata stays small and fast to query, S3 handles objects of any size, and the item holds a pointer.

THE SIGNAL

'Intermittent failures on larger items' names the 400 KB ceiling without saying it. Large object in S3, pointer in DynamoDB, is the answer AWS documents.

A billing table holds about 900 million invoice items. A nightly job must find every invoice in the DISPUTED state and route it for review. Disputed invoices number in the low thousands at any time, and the flag is cleared once an invoice is resolved.

The current job scans the entire table each night, takes over four hours, and consumes most of the account's read capacity.

What is the most efficient way to find the disputed invoices?

A Create a GSI on a disputedAt attribute that is written only while an invoice is disputed, and removed when resolved

B Create a GSI with status as the partition key and query for DISPUTED

C Keep the scan but add a filter expression on status and run it in parallel segments

D Enable DynamoDB Streams and maintain a separate list of disputed invoice IDs in ElastiCache

◆ PAUSE AND DECIDE

What is the most efficient way to find the disputed invoices?

A Sparse GSI on disputedAt

Only disputed invoices carry the attribute, so the index holds a few thousand items instead of 900 million. Removing the attribute on resolution removes the item from the index.

B GSI on status

Works, and it indexes all 900 million items, duplicating storage and adding a write to every invoice change. It also makes DISPUTED a low-cardinality partition key.

C Parallel scan with a filter

Faster in wall-clock time, and it still reads all 900 million items and consumes the same capacity. Filters do not reduce what is read.

D Streams into ElastiCache

Builds and operates an index by hand, adds a cache to keep consistent, and loses the data if the cache is flushed.

THE SIGNAL

Compare with scenario 2, where a full GSI on status was correct. Here the target set is a millionth of the table and the flag is cleared - which is exactly when sparse wins.

What to carry into DynamoDB II

- Model the **access patterns** first. The primary key cannot be changed later.
- Only the partition key spreads load. Per-partition caps are ~3,000 RCU and ~1,000 WCU.
- **Throttling with capacity to spare** means a hot partition, not a capacity shortage.
- **LSI only for strong consistency, and only at table creation.** Otherwise GSI.
- Sparse indexes find a small set in a huge table without ever scanning.

> ON THE EXAM
>
> Next: reading and writing. Query against Scan, consistency, conditional writes, transactions, TTL, capacity modes and DAX.